

1. Bối cảnh & Dự đoán sơ bộ

- **Triệu chứng:** Có thể tính chạy rất chậm (lag, đơ), cột **Bộ nhớ** trong Trình quản lý tác vụ **95%** mặc dù chỉ mở hai ứng dụng phổ biến ở đó Google Chrome đi tới VS Code.
- **Nguyên nhân cốt lõi:** Sự hợp lý giữa tài nguyên tiêu tốn quá mức từ các ứng dụng (Chrome mở nhiều tab/tiện ích mở rộng nặng, VS Code chạy các (RAM).

2. Giải thích tình huống qua mô hình “Ban quản lý tòa nhà”

Để dễ dàng hình dung cách hệ điều hành vận hành, hãy tưởng tượng hệ thống máy tính như một tòa nhà văn phòng:

- **Phần cứng (Phần cứng - CPU, RAM, Ổ cứng):** Tòa nhà vật lý bao gồm các công việc phòng (RAM) và kho chứa tài liệu (Storage).
- **Kernel (Nhân hệ điều hành): Ban quản lý tòa nhà** . Đây là cơ quan đầu tiên nắm giữ chìa khóa toàn bộ các ốc, điều tiết nguồn điện, phân tích bổ sung không gian làm việc và đảm bảo an ninh, an toàn cho hệ thống.
- **Shell (Giao diện người dùng/Lề tân): Bộ phận lễ tân** . Nhận yêu cầu từ người thuê văn phòng (User) rồi chuyển xuống cho Ban quản lý (Kernel) thực thi.
- **Quy trình (Chrome, VS Code): Các công ty thuê phòng** .
 - *Tình huống hiện tại:* Công ty này mang quá nhiều nhân viên và tài liệu vào phòng (chiếm 95% RAM), khiến các công việc trong phòng được quá tải, nhân viên không còn chỗ ngồi, di chuyển khó khăn (gây hiện tượng giật lag).

3. Luồng dữ liệu theo IPO mô hình (Tải ứng dụng từ Bộ lưu trữ vào RAM)

Khi bạn thực hiện thao tác mở VS Code từ ổ cứng lên màn hình, hệ điều hành xử lý qua 3 bước:

- **Đầu vào (Đầu vào):** Nhấn đúp vào ứng dụng qua Shell. Đây là nơi chứa Shell Kernel.
- **Quy trình (Xử lý):**
 - Nhân hệ điều hành được lưu trữ trong bộ nhớ RAM.
 - Kernel tạo một khối quản lý tiến trình (**PCB** - Process Control Board) để giám sát ứng dụng.
 - Kernel ra lệnh đọc chương trình dữ liệu từ ổ cứng (Bộ lưu trữ) vào ô nhớ (khối RAM) đã được cấp phát.
- **Đầu ra (Đầu ra):** Ứng dụng được tải xuống thành công vào RAM, CPU bắt đầu thực thi các nguồn mã lệnh của quy trình và ứng dụng giao diện hiển thị lên màn hình cho người dùng tương tác.

4. Xử lý cưỡng bức & đánh lừa kỹ thuật

A. Điều gì sẽ xảy ra nếu cố gắng hoàn thành quá trình “End Task” **System**?

- **Hậu quả:** Tiến trình **System**(hoặc **ntoskrnl.exe**) là linh hồn của Kernel, chịu trách nhiệm quản lý các hoạt động cấp thấp của hệ thống. (End Task) **màn hình xanh (BSOD - Blue Screen of Death)** và bắt buộc phải khởi động lại.

B. Bạn có thể lựa chọn giữa SSD và HDD với RAM (Bộ nhớ ảo / Bộ nhớ hoán đổi)

Khi RAM đầy (95%), hệ điều hành phải sử dụng một phần cứng làm RAM ảo (**Swap file** hoặc **Pagefile**) để chứa ít dữ liệu tạm thời được sử dụng.

- **Nếu máy dùng SSD:** Tốc độ đọc/ghi tốc độ cao của SSD (từ 500 MB/s đến vài GB/s). Việc thay đổi bộ nhớ (phân trang) diễn ra nhanh hơn, máy vẫn chậm nhưng vẫn có thể phản hồi tác vụ sau một chút.
- **Nếu máy dùng HDD:** Tốc độ đọc/ghi rất chậm (chỉ khoảng 80–140 MB/s) do start from the basic learning phải chuyển liên tục. Khi RAM đầy và chuyển sang dùng Swap trên HDD, ổ cứng sẽ được tăng **100% Disk Usage** , gây ra hiện tượng **Thrashing** (treo toàn diện), máy gần như bất động và không thể thao tác gì thêm.

5. Báo cáo phân tích chuyên sâu

Quản lý bộ nhớ (Quản lý bộ nhớ)

Hệ điều hành sử dụng bộ nhớ phát hiện cơ chế (Paging). Chrome không có cùng RAM với tiện ích mở rộng RAM (Process Isolation). Kết hợp với VS Code (chạy Node.js và cú pháp phân tích tiện ích mở rộng), tổng dung lượng bộ nhớ cần phát hiện vượt quá giới hạn của RAM, ép hệ thống phải sử dụng Pagefile gây suy giảm hiệu ứng nghiêm trọng.

Quản lý tiến trình (Process Management)

Bộ cài đặt lịch CPU (CPU Scheduler) phải phân chia thời gian xử lý cho hàng loạt tiến trình của Chrome và VS Code. Khi RAM cạn kiệt, hệ điều hành tốn kém quá nhiều thời gian chỉ để quản lý việc chuyển đổi dữ liệu giữa RAM và ổ cứng (thay vì (Thời gian đáp ứng).

6. Sơ đồ hệ thống tầng tầng (Architecture Layers)

Sơ đồ có thể hiển thị luồng tương tác từ người dùng xuống phần cứng khi mở một ứng dụng:

Văn bản thuần túy

